

Functions - Questions

Question: Functions 1 - Code Readability

Short Humorous Scenario: Martin har seen some code that works but is hard til read. *"This function1.c program prints element counts, but it's so hard til understand! What would help make this more readable? The code must not change functionality."* han spørger.

Hvad de allerede ved:

- Comments in C: `//` og `/* */`
- Variabel naming conventions
- Funktion structure
- `printf` formatting

Opgaven:

Given the program output:

```
N(211)=15  
N(93)=9
```

1. Examine the code in `functions1.c`
2. Identify what would improve readability
3. Answer: What helps make code more readable without changing functionality?
4. **PBL:** Martin spørger: *"Kan you explain why meaningful variable names are important in programming?"*

Hint:

- Meaningful variable names (e.g., `values_above_element` instead of `x`)
- Comments explaining non-obvious code
- Consistent indentation
- Proper spacing
- Se the code in `functions1.c`

Hvad du aldrig må gøre:

- Change the program's functionality
- Remove necessary code
- Alter the logic

Question: Functions 2 - BMI Calculation Function

Short Humorous Scenario: Martin ønsker at calculate BMI but doesn't want the formula in main(). *"Jeg skal a function that calculates BMI. Kan you make one?"* han spørger. *"It should take weight og height, og return the BMI."*

Hvad de allerede ved:

- Funktion definition og declaration
- Funktion parameters
- Returner values
- BMI formula: `bmi = weight / (height * height)`

Opgaven:

1. Definer variables `weight = 31.1` og `height = 1.31` i `main()`
2. Definer function `float calculateBMI(...)` over `int main()`
3. In the `calculateBMI` function:
 - Implement the BMI algorithm
 - Returner the BMI value
4. In `main`, call `calculateBMI` with `height` og `weight` as input parameters
5. Gem the returned value in a `float bmi` variable
6. Udskriv the result
7. **PBL:** Martin spørger: *"Kan you modify this function til also print a category (Underweight, Normal, Overweight, Obese) based on the BMI?"*

Hint:

- What datatypes should input parameters be? `float` for both
- What should the return type be? `float`
- Funktion signature: `float calculateBMI(float weight, float height)`
- Call: `float bmi = calculateBMI(height, weight);`
- BMI categories: Underweight < 18.5, Normal 18.5-24.9, Overweight 25-29.9, Obese >= 30

Hvad du aldrig må gøre:

- Change the BMI calculation formula
- Glem til declare function before use
- Brug wrong datatypes for parameters

Question: Functions 3 - Udskriv Value with Pointer

Short Humorous Scenario: Martin ønsker at print a variable's memory address og value. *"Jeg skal a function that takes a pointer til an int og prints the address og value. Kan you make this?"* han spørger.

Hvad de allerede ved:

- Pointers
- Funktion parameters (pointer types)
- Address-of operator `&`
- Dereference operator `*`
- Format specifier `%p` for addresses

Opgaven:

1. Definer variable `int value = 101;` in `main`
2. Definer function `void printValue(int *pval)` above `main`
3. In `printValue` function:
 - Implement the print function til show: `address: 0xF213d23, value: 101`
4. Kald `printValue(&value)` from `main`
5. What information is passed til `printValue`?
6. Kør program
7. **PBL:** Martin spørger: "Kan you modify this til also print the size of the pointer in bytes?"

Hint:

- Hvordan til print a memory address: use `%p` format specifier
- Hvordan til dereference a pointer: use `*` (e.g., `*pval`)
- Pass address using `&`: `printValue(&value)`
- The information passed is the memory address (pointer value)

Hvad du aldrig må gøre:

- Pass the value instead of the address
- Glem til dereference in the function
- Brug wrong format specifier for address

Question: Patient 1 - Udskriv Funktion (Call-by-Value)

Short Humorous Scenario: Martin ønsker at organize his patient printing code. "Jeg har patient information til print, but it's all in `main()`. Kan you make a function til handle this?" han spørger.

Hvad de allerede ved:

- Funktion definition og declaration
- Funktion parameters (call-by-value)
- `printf` function
- Multiple parameters

Opgaven:

1. Definer en function declaration og definition `printPatientInfo(...)`

2. Start slowly with 1 single parameter: `id` (lines 11-12)
3. Replace `printf` lines in `main` with `printPatientInfo` function calls
4. Note: `int main` should not contain any `printf` calls

Starting code:

```
#include <stdio.h>
int main() {
    int id = 13;
    int age = 44;
    float weight = 80;
    float height = 1.80;
    printf("Patient Information:\n");
    /* Udskriv patient information */
    printf("###\n");
    printf(" PatientID:\t%d\n", id);
    printf(" age:\t\t%d years\n", age);
    printf(" weight:\t%.1f kg\n", weight);
    printf(" height:\t%.2f m\n", height);
    return 0;
}
```

5. **PBL:** Martin spørger: "Kan you add all patient parameters til the function og remove all `printf` from `main`?"

Hint:

- Start med just `id` parameter first
- Tilføj derefter `age`, `weight`, `height` én ad gangen
- Funktion should contain all the `printf` statements
- Call: `printPatientInfo(id, age, weight, height);`

Hvad du aldrig må gøre:

- Lad nogen `printf` være i `main`
- Glem function declaration
- Brug wrong parameter order

Question: Patient 2 - Multiple Patients with BMI

Short Humorous Scenario: Martin ønsker at print multiple patients with BMI calculation. "Jeg har an array of patients. Kan you use the `print` function og add BMI calculation?" han spørger.

Hvad de allerede ved:

- Arrays
- Functions
- Loops
- BMI calculation

Opgaven:

1. Brug `printPatientInfo` code from `patient1.c`
2. Lav a function `calculateBMI` that calculates $bmi = weight / (height * height)$
3. Modify `printPatientInfo` til print BMI

Starting code:

```
#include <stdio.h>
int main() {
    int ids[] = {1, 3, 52, 23};
    int ages[] = {13, 44, 51, 100};
    float weights[] = {80, 99, 100, 50};
    float heights[] = {1.80, 1.62, 1.30, 1.74};
    int number_of_patients = sizeof(ids) / sizeof(ids[0]);
    printf("Patient Information for %d patients\n", number_of_patients);
    for (int i = 0; i < number_of_patients; i++) {
        /* Udskriv patient information */
        printf("###\n");
        printf(" PatientID:\t%d\n", ids[i]);
        printf(" age:\t\t%d years\n", ages[i]);
        printf(" weight:\t%.1f kg\n", weights[i]);
        printf(" height:\t%.2f m\n", heights[i]);
    }
    return 0;
}
```

4. **PBL:** Martin spørger: "Kan you also print the BMI category for each patient?"

Hint:

- Remember function declaration og definition of `printPatientInfo`
- Beregn BMI: $bmi = calculateBMI(heights[i], weights[i])$
- Udskriv BMI with: `printf(" BMI:\t\t%.1f\n", bmi);`

Hvad du aldrig må gøre:

- Beregn BMI i main
- Glem til call `calculateBMI`
- Glem til print BMI

Question: Records Display - Call-by-Reference

Short Humorous Scenario: Martin ønsker at print struct data using a function. "Jeg har a Record struct with *id*, *date*, *notes*, og *measurement*. Kan you make a function that prints it?" han spørger.

Hvad de allerede ved:

- Struct definition
- Funktion parameters (call-by-reference)

- `ctime()` function for date formatting
- Pointers til structs

Opgaven:

1. Definer en struct template for `Record` der indeholder:
 - `id`: integer
 - `dato`: `time_t`
 - `notes`: 100 char string
 - `measurement`: float
2. Lav function declaration og definition `print_record(Record *)` (call-by-reference)
3. The function should:
 - Udskriv a readable date from `dato` (see `ctime(t)`)
 - Example output for 1594974180: `date: Fri Jul 17 10:23:00 2020`
 - Udskriv all other information
4. In `main`, define:

```
struct Record rec = {131, 1594974180, "Measurement recorded without  
issues", 93.3};
```

5. Kald `print_record` with `rec` as call-by-reference
6. Kør program
7. **PBL:** Martin spørger: *"Can you extend this til print multiple records from an array?"*

Hint:

- Pass struct address: `&rec` or `rec`
- Brug `ctime(&record.dato)` til get readable date
- Inkluder `time.h` for `ctime()`
- Funktion signature: `void print_record(struct Record *r)`

Hvad du aldrig må gøre:

- Pass struct by value (makes a copy)
- Glem til include `time.h`
- Glem & when passing struct

Question: Patient 3 - Statistics Functions

Short Humorous Scenario: Martin ønsker at calculate statistics for multiple patients. *"Jeg skal mean og standard deviation functions. Can you make them reusable?"* han spørger.

Hvad de allerede ved:

- Arrays
- Functions
- Loops
- Statistical formulas
- Math library

Opgaven:

1. Skriv function `printStatistics(char* measurement, float gennemsnit, float afvigelse)` - it should print measurement, mean, og std
2. Skriv function `float mean(float x[], int n)` - calculates mean $\mu = \frac{1}{N} \sum_{t=0}^{n-1} x_t$
3. Skriv function `float std(float x[], int n)` - calculates std $\sigma = \sqrt{\frac{1}{N-1} \sum_{t=0}^{n-1} (x_t - \mu)^2}$
4. Skriv og use `printStatistics` function

Starting code:

```
#include <stdio.h>
int main() {
    int ages[] = {13, 44, 51, 100};
    float weights[] = {80, 99, 100, 50};
    float heights[] = {1.80, 1.62, 1.30, 1.74};
    int number_of_patients = sizeof(ages) / sizeof(ages[0]);
    printf("Patient Descriptives for %i Patients\n", number_of_patients);
    return 0;
}
```

5. **PBL:** Martin spørger: "Kan you modify this til calculate og print statistics for all three measurements (age, weight, height) in one loop?"

Hint:

- You've made these equations before
- Mean: $\mu = \frac{1}{N} \sum x_t$
- Std: $\sigma = \sqrt{\frac{1}{N-1} \sum (x_t - \mu)^2}$
- Husk at inkludere `<math.h>` for `sqrt()`

Hvad du aldrig må gøre:

- Brug N instead of N-1 for std
- Glem til include math.h
- Beregn mean/std incorrectly